

可移植算子 · 训推优化

正确性、稳定性、性能：可移植算子交付的三张验收表

算子适配

图融合

长跑稳定性

Benchmark

岗位能力画像

岗位画像：面对不同硬件/编译器生态时，考察库路径、定制 kernel、fallback、长跑稳定性和测试矩阵。

高频深挖：题目背后的判断

新平台怎么先跑起来？

优先复用成熟库/编译器路径；缺覆盖时再给定制 kernel，并始终保留安全 fallback。

library → custom kernel → fallback。

静态图 fusion 先看什么？

确认数据依赖、旁路消费者、shape/dtype、数值语义；不要把相邻节点默认当作可融合。

少一次中间写回，不应换来失控的资源压力。

随机崩溃怎样分层？

先查 host/device placement、shape tensor、生命周期和异步错误，再看长期负载。

一次通过不等于稳定。

benchmark 怎样可信？

warmup、同步口径、多次采样、shape×dtype×batch、正确性 diff 和 fallback 覆盖缺一不可。

性能数据必须有范围与对照。

项目叙事的通用深挖链

问题与约束

证据与机制

方案与取舍

验证与边界

答题提醒

不要从‘用了什么库/指令’开始；先固定工作负载、指标和约束，再说明数据流/资源/依赖，最后给出正确性、性能和反例边界。

把一句结论变成可复验的事实

现象

固定 workload / 阶段 / 指标，明确‘慢’或‘错’在哪里。

证据

用 trace、counter、对照或最小复现排除相邻假设。

机制

说明资源、数据流或依赖如何导致这个现象。

验收

数值、边界、重复性和 E2E 共同成立后再下结论。

一题答透：算子交付路径：快路径之外，必须有安全路径

问题 “如何把一个新算子从能跑交付到可长期使用？”

1. 语义

明确输入/输出、broadcast、layout、dtype、边界与 CPU fallback 是否等价。

2. 实现

先接成熟库；自定义路径只覆盖条件明确的 shape；其余交给 fallback。

3. 正确性

单测、数值 diff、边界 shape、race/mem check；错误路径也要被触发。

4. 稳定与性能

长跑、恢复、图模式、真实 shape benchmark 与 E2E 回归分别记录。

把推理链画出来



常见误区

× 把 CPU fallback 的结果当作 GPU 性能。

× 只跑一次 benchmark，没有 warmup、同步和重复。

× 修了随机崩溃却没有长跑或资源检查。

术语速懂 | 面试怎么答

术语速懂

Fast path + Fallback

高性能 kernel 只覆盖满足 shape/layout/dtype 条件的输入；fallback 是显式正确性安全网。

Host/device placement

数据 tensor 与 shape/控制 tensor 生命周期不同；错误放置会形成异步错误或随机崩溃。

面试中怎么答

- ① 先定义算子语义与快路径覆盖边界，再谈性能。
- ② 用 selector 明确分流，沿数据流和控制流排查 placement。
- ③ 分别验数值、图模式、动态 shape、长跑和资源检查。

白板 / 手撕：白板题：broadcast / elementwise 与融合边界

题面 设计一个带 scalar broadcast 的逻辑/逐元素算子，并讨论如何安全融合到相邻 activation。

CUDA C++ · broadcast + activation fusion

核心实现

```
01 __global__ void bias_relu(const float* x, const float* bias,
02     float* y, int rows, int cols) {
03     size_t i = size_t(blockIdx.x) * blockDim.x + threadIdx.x;
04     size_t n = size_t(rows) * cols;
05     if (i >= n) return;
06     int col = i % cols; // last-dim broadcast
07     y[i] = fmaxf(0.f, x[i] + bias[col]);
08 }
```

写代码时怎么讲

1. 先写 index 映射、broadcast 规则、dtype 和异常/NaN 语义。
2. global 访问要求连续/coalesced；向量化需对齐与 tail mask。
3. fusion 前确认中间输出没有旁路消费者，并评估 register/smem 与 occupancy。
4. 测试包括 scalar、不同 rank、非连续 shape、极端长度和 CPU reference。

实现重点

- 先写清 row-major、连续布局、broadcast 维度与 NaN 语义。
- $i < n$ 是尾部 mask；非连续输入应走安全 fallback。
- 中间张量有旁路消费者或资源压力过大时不应强行融合。

边界、测试与追问

最少要说明的验证

- 先写清 row-major、连续布局、broadcast 维度与 NaN 语义。
- $i < n$ 是尾部 mask；非连续输入应走安全 fallback。
- 中间张量有旁路消费者或资源压力过大时不应强行融合。

面试继续追问

- 怎样避免把 shape tensor 错放到 device？
- 图模式与 eager 模式应分别验什么？
- 如何让 fallback 不掩盖新路径错误？